

FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P1

Relatório bimestral de atividades

Avaliação individual · 2026.2

Yago Felipe de Oliveira Martins

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 07/10/2026

Sumário

1	Introdução	2
2	Metodologia	2
3	Quadro de entregas	2
4	Atividades realizadas	3
4.1	Atividade 1 — Agenda telefônica em Flask	3
4.2	Atividade 2 — Catálogo de filmes — Tom Hanks	3
4.3	Atividade 3 — Desacoplando o login — microsserviço de autenticação	6
4.4	Atividade 4 — Controle de acesso por papel — RBAC	9
4.5	Atividade 5 — Logs e auditoria	12
4.6	Atividade 6 — Upload e perfil de usuário	15
5	Considerações finais	18
6	Declaração de autoria	18

1 Introdução

O presente relatório é elaborado no âmbito da disciplina de Introdução à Computação em Nuvem, integrante da estrutura curricular do curso superior de Sistemas Inteligentes. A disciplina propõe-se a introduzir e consolidar conceitos fundamentais de arquitetura de software, desacoplamento de serviços, armazenamento de objetos e gestão de infraestrutura containerizada na nuvem, aplicando práticas modernas do ecossistema de microsserviços.

Como fio condutor das atividades do bimestre, desenvolveu-se a aplicação “Catálogo Tom Hanks”, uma plataforma web para consumo e exibição de dados de filmes integrando a API externa do TMDb. O projeto evoluiu de uma estrutura inicial para uma arquitetura desacoplada e escalável baseada em microsserviços geridos por Docker e Portainer, incorporando persistência de dados com MariaDB, armazenamento de imagens de perfil no MinIO e registo centralizado de logs de auditoria através do Redis Streams.

Neste relatório, descrevem-se detalhadamente as etapas de concepção, implementação e refinamento do sistema. A documentação abrange o planeamento do Controle de Acesso Baseado em Papéis (RBAC) para usuários e administradores, a gestão de uploads e políticas de leitura do MinIO, os mecanismos de auditoria no log-service, bem como os desafios técnicos enfrentados na interface do usuário.

2 Metodologia

Todas as atividades foram desenvolvidas no mesmo repositório público do GitHub, como continuação de um único projeto (catálogo de filmes). A data e a hora de cada entrega foram extraídas do histórico de commits (git log) e conferidas na página do commit no GitHub. Os prints mostram o README com a menção ao professor, o histórico de commits e o sistema em execução.

3 Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.	07/08/2026	Não realizada	não entregue
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.	20/08/2026	20/08/2026 21:38	entregue
3	Desacoplando o login — microsserviço de autenticação	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.	28/08/2026	28/08/2026 13:57	entregue
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	02/09/2026 00:32	entregue
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	25/09/2026 12:45	entregue

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
6	Upload e perfil de usuário	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.	02/10/2026	02/10/2026 17:17	entregue

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.
Data planejada	07/08/2026
Data realizada	Não realizada
Situação	não entregue
Evidência	Realizada em sala — print do sistema rodando localmente (e do repositório, se foi versionado)
Links	• Repositório do Projeto

O que foi feito. A Atividade 1 consistia no nivelamento prático em sala de aula com a construção de uma aplicação monolítica em Flask utilizando Jinja para renderização de templates e persistência de dados em arquivos JSON. A proposta da atividade era servir como base conceitual de arquitetura e código para os projetos que seriam desenvolvidos ao longo do bimestre.

Dificuldades e como foram resolvidas. A atividade consta como não entregue devido à minha ausência na aula prática do dia 07/08/2026, data em que o exercício foi realizado e orientado em sala. Por conta dessa falta, o projeto não pôde ser desenvolvido no modelo proposto. A limitação foi superada nas etapas seguintes através do estudo autônomo da documentação e da construção direta do “Catálogo Tom Hanks” do zero, estruturando a aplicação e suas dependências sem a utilização do modelo base da agenda.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDB, persistência em MariaDB e segregação por usuário.
Data planejada	20/08/2026
Data realizada	20/08/2026 21:38
Situação	entregue
Evidência	GitHub — commit + README + print do catálogo
Links	• Repositório • Issue 1 • README • Commits

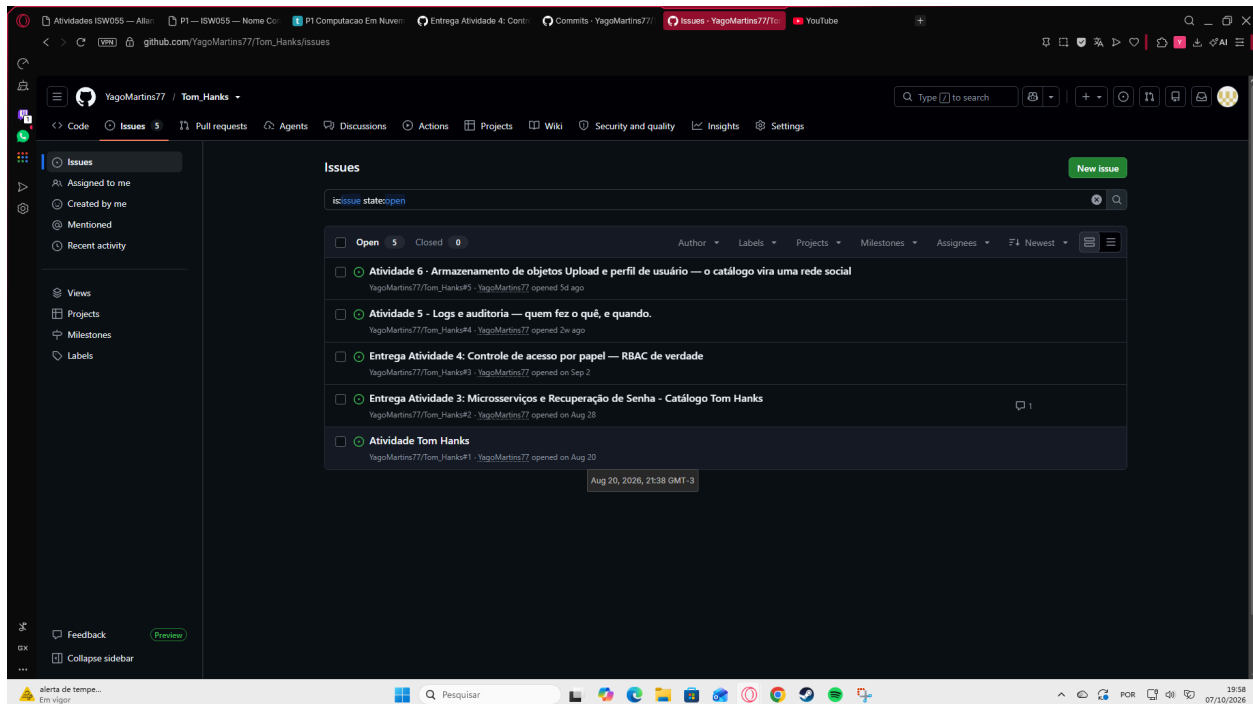
O que foi feito. Desenvolveu-se a versão inicial do “Catálogo Tom Hanks”, integrando a aplicação à API pública do TMDB (The Movie Database) para a busca e exibição dos filmes do ator. Implementou-se a

camada de persistência de dados utilizando o banco de dados relacional MariaDB para permitir que os usuários pudessem favoritar filmes e registrar comentários. O sistema foi projetado para garantir a segregação completa dos dados por usuário, assegurando que cada pessoa visualize e gerencie individualmente sua lista de favoritos e seus próprios comentários. A infraestrutura básica foi configurada em ambiente containerizado com Docker, estabelecendo as fundações da aplicação para as evoluções futuras.

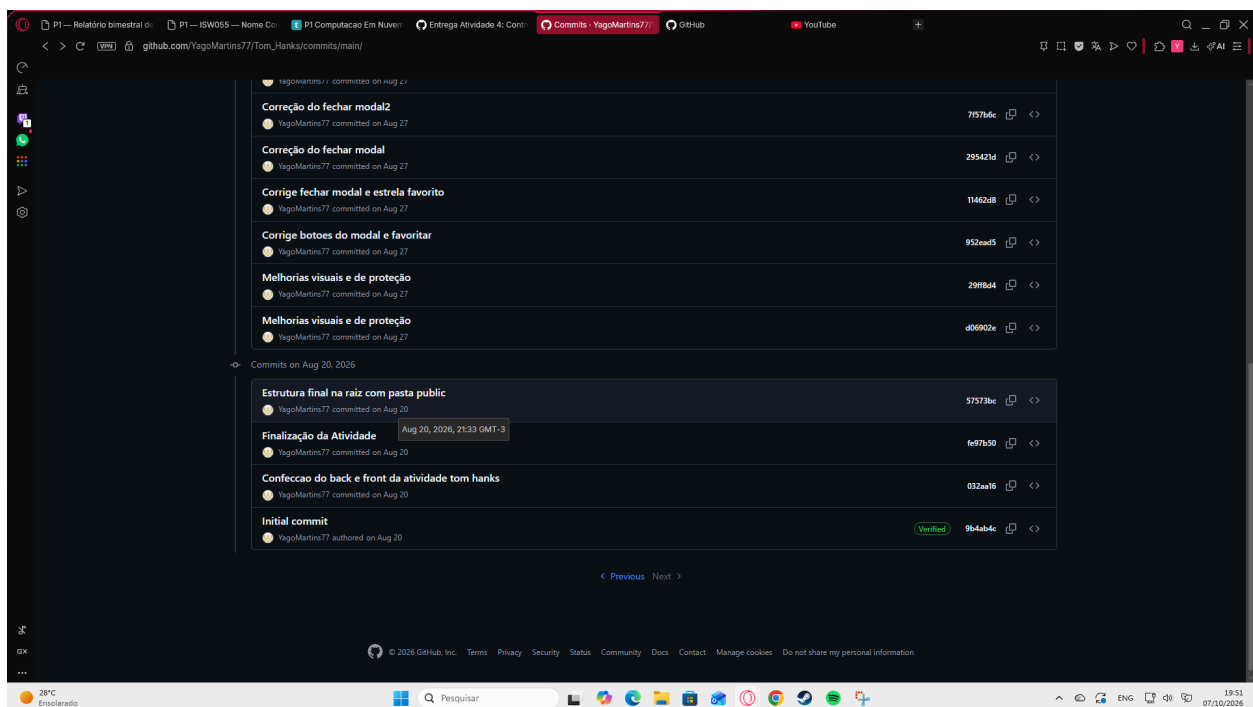
```
├── .env.example
├── .gitignore
├── Dockerfile
├── README.md
├── package-lock.json
├── package.json
├── public/
│   ├── auth.html
│   ├── index.html
│   └── style.css
└── server.js
```



“Interface do Catálogo Tom Hanks listando filmes consumidos da API do TMDb.”



“Evidência da Issue 1 no GitHub comprovando o registro e a data/hora da entrega da Atividade 2.”



“Evidência de commit no GitHub para a Atividade 2 (Catálogo MariaDB) com hash, data e hora visíveis.”

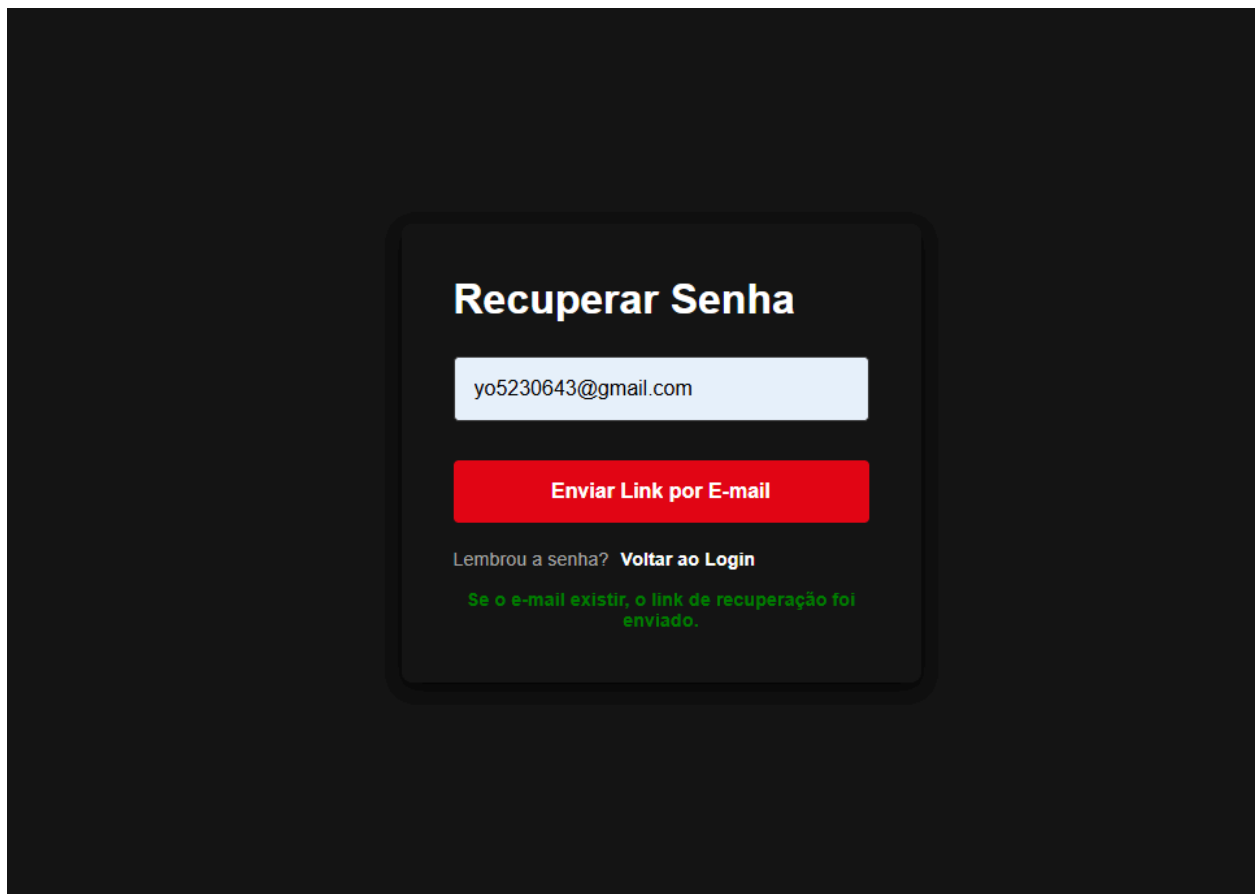
Dificuldades e como foram resolvidas. Devido à ausência na primeira aula prática — ocasião em que foi desenvolvida a Atividade 1 (Agenda Telefônica em Flask), utilizada como modelo base para o projeto —, o principal desafio inicial consistiu em estruturar a aplicação do zero sem o template prévio fornecido em sala. Para superar essa limitação, realizou-se um estudo autônomo da documentação do ecossistema e dos padrões de arquitetura necessários.

4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026
Data realizada	28/08/2026 13:57
Situação	entregue
Evidência	GitHub — commit + docker-compose.yml + print do login funcionando
Links	• Repositório • Issue 2 • README • Commits

O que foi feito. Nesta atividade, a arquitetura da aplicação foi evoluída para um modelo desacoplado através da criação do auth-service, um microsserviço dedicado exclusivamente ao gerenciamento de autenticação, registro de usuários, controle de papéis (role) e recuperação de senha. O serviço foi isolado em um container próprio rodando na rede interna do Docker (app-network), operando sem portas expostas diretamente para a internet. Toda a comunicação com o cliente passou a ser intermediada pelo catalogo-service, que atua como ponto de entrada público e repassa as requisições de autenticação internamente para o endereço <http://auth-service:3001>.

Para a funcionalidade de redefinição de senha, implementou-se a integração com o Mailtrap para o envio de e-mails de recuperação contendo tokens de uso único com tempo de expiração de 30 minutos, armazenados na tabela reset_tokens. Essa decisão garantiu maior segurança ao processo e consolidou a separação de responsabilidades no ecossistema de microsserviços.



Tela de login e cadastro processada pelo microserviço desacoplado auth-service.“

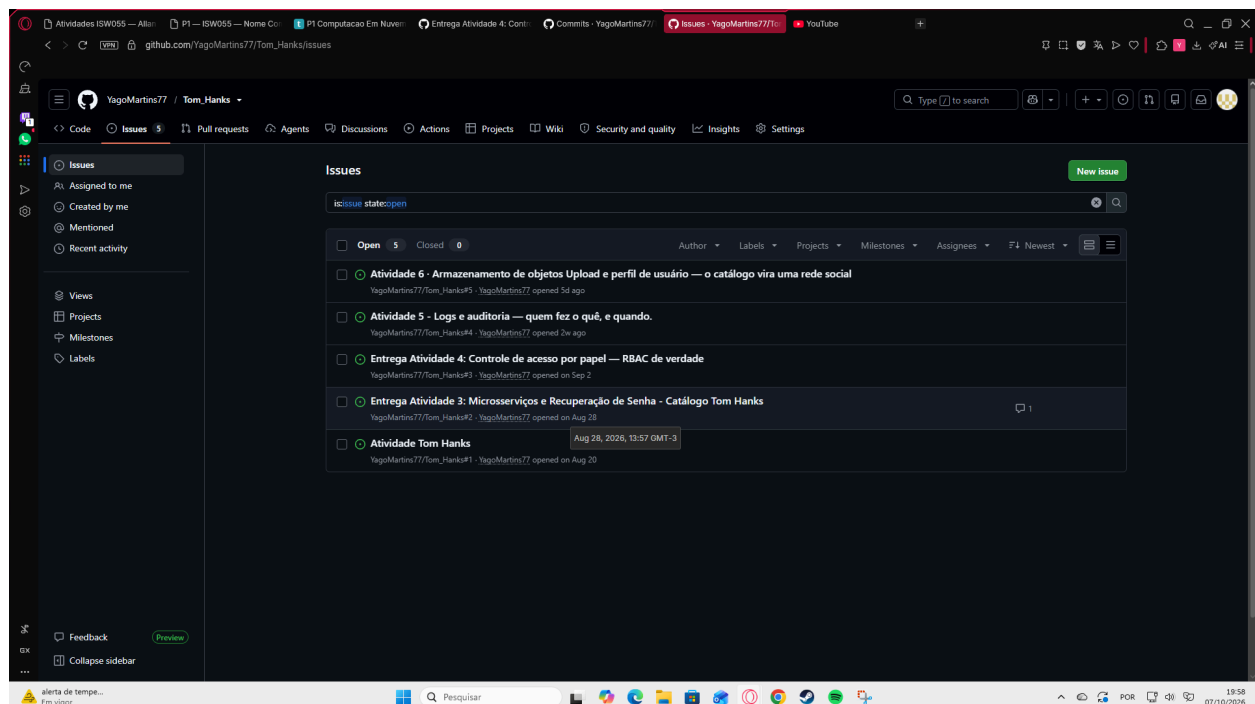


“Formulário de solicitação de redefinição de senha de usuário.”

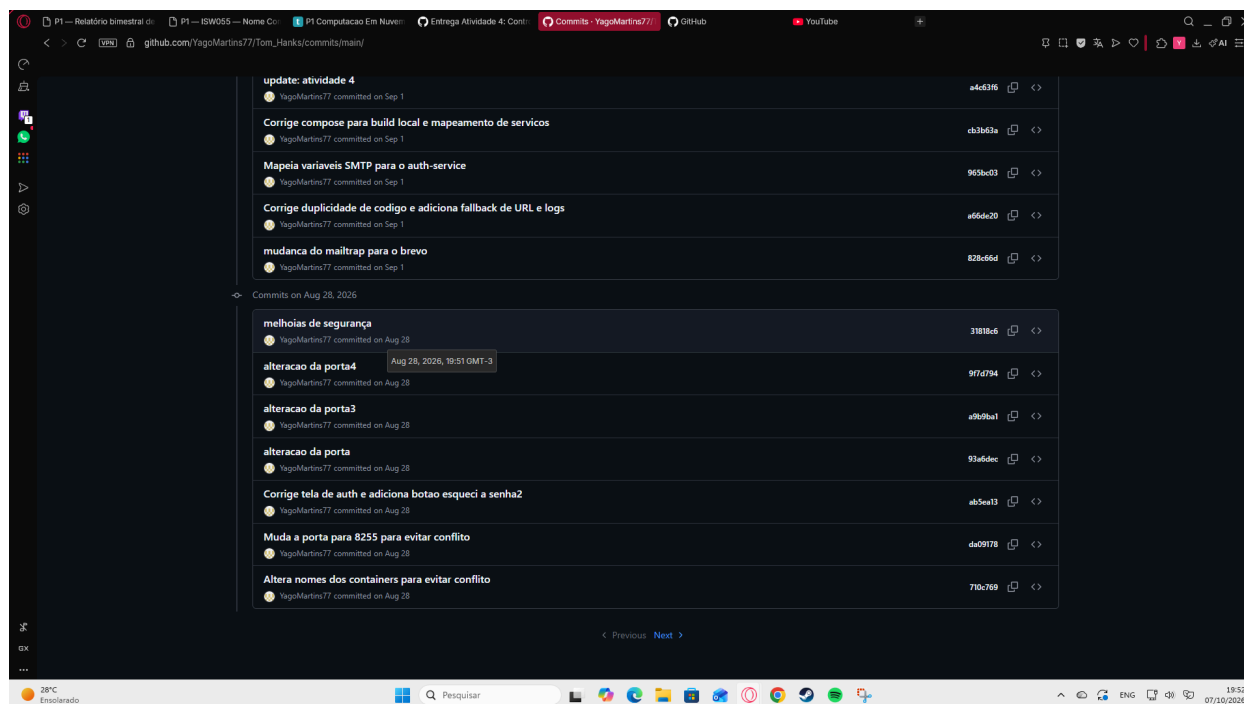


O formulário de redefinição de senha apresenta um fundo escuro com o título "Nova Senha" em branco. Abaixo do título, há um campo de entrada de texto cinza com o placeholder "Digite a nova senha (mínimo 6 caractere)". Na base do formulário, há dois botões: um botão principal vermelho com o texto "Salvar Nova Senha" e um botão secundário branco com o texto "Voltar para Login".

“E-mail de recuperação de senha com token recebido no Brevo(Como não foi tirado o print no dia da entrega, acabou que para fazer a presente atividade foi necessário tirar um print com o sistema de alteração de senhas ja funcionando via brevo).”



“Evidência da Issue 2 no GitHub comprovando o registro e a data/hora da entrega da Atividade 3.”



“Evidência de commit no GitHub para a Atividade 3 (Desacoplamento do auth-service) com hash, data e hora visíveis.”

Dificuldades e como foram resolvidas. A principal dificuldade enfrentada durante esta etapa esteve relacionada ao envio dos e-mails de recuperação de senha, que não chegavam devido a um conflito de configurações e rotas no arquivo server.js dentro da pasta auth-service. O problema foi resolvido após uma revisão detalhada do código do servidor, ajustando o fluxo de envio da mensagem e corrigindo os parâmetros de conexão do serviço de e-mail na rotina interna do serviço.

4.4 Atividade 4 — Controle de acesso por papel — RBAC

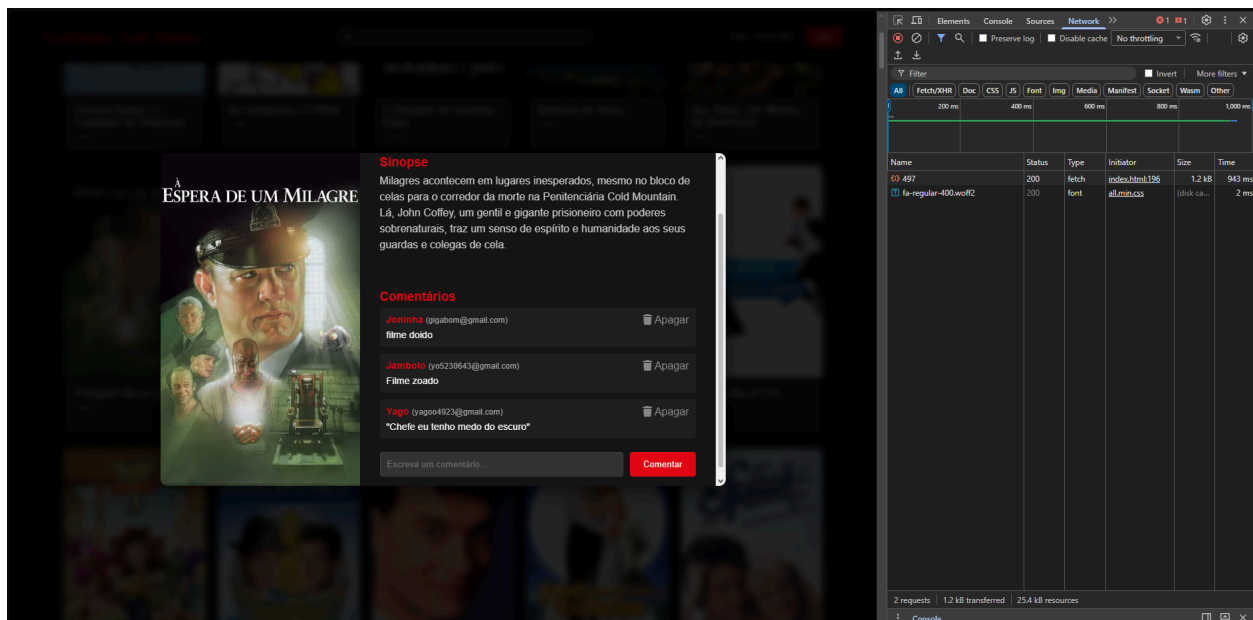
Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	02/09/2026 00:32
Situação	entregue
Evidência	GitHub — commit + print do 403 e da ação de admin
Links	• Repositório • Issue 3 • README • Commits

O que foi feito. Implementou-se o modelo de Controle de Acesso Baseado em Papéis (Role-Based Access Control - RBAC) no backend para garantir a aplicação de permissões no sistema. O campo role (usuario ou admin) associado à sessão/token passou a validar o acesso a rotas e ações sensíveis. Usuários comuns têm permissão para visualizar o catálogo, autenticar-se, favoritar filmes, adicionar comentários e excluir apenas os seus próprios comentários. Já o usuário com o papel admin possui privilégios de moderação global, conseguindo visualizar e excluir comentários de qualquer usuário, além de acessar funcionalidades

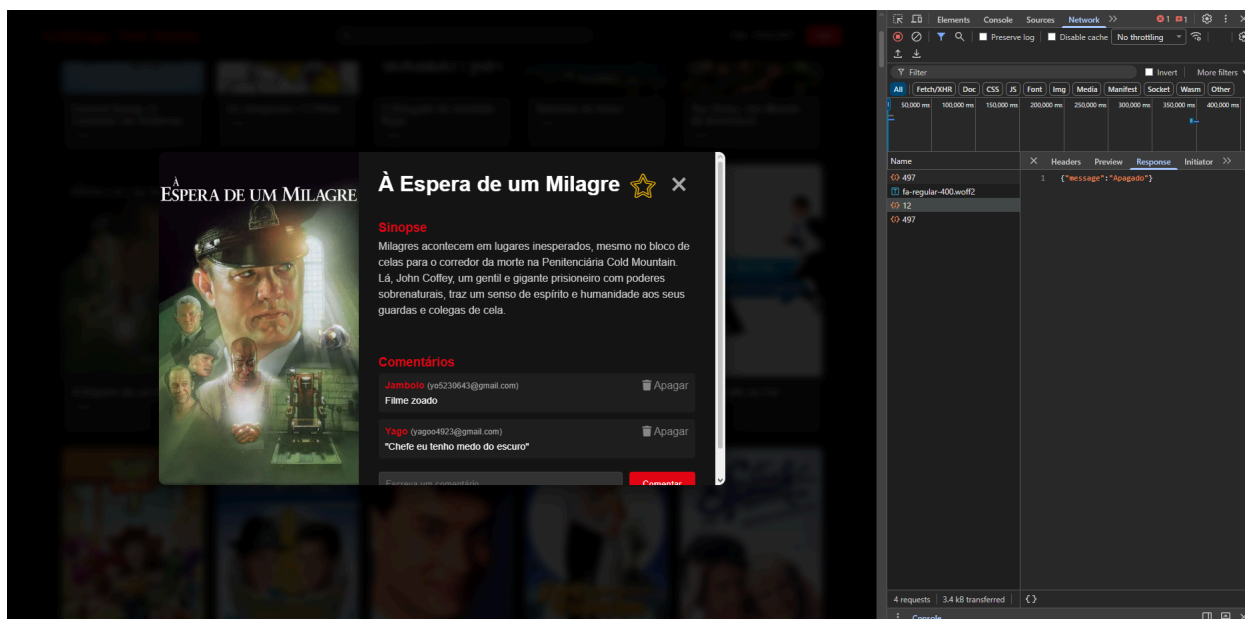
administrativas restritas. Tentativas de acesso não autorizado por usuários comuns resultam em resposta com código HTTP 403 (Forbidden). Adotou-se o Padrão A (Enforcement Centralizado), consultando as permissões diretamente no backend a cada requisição sensível.



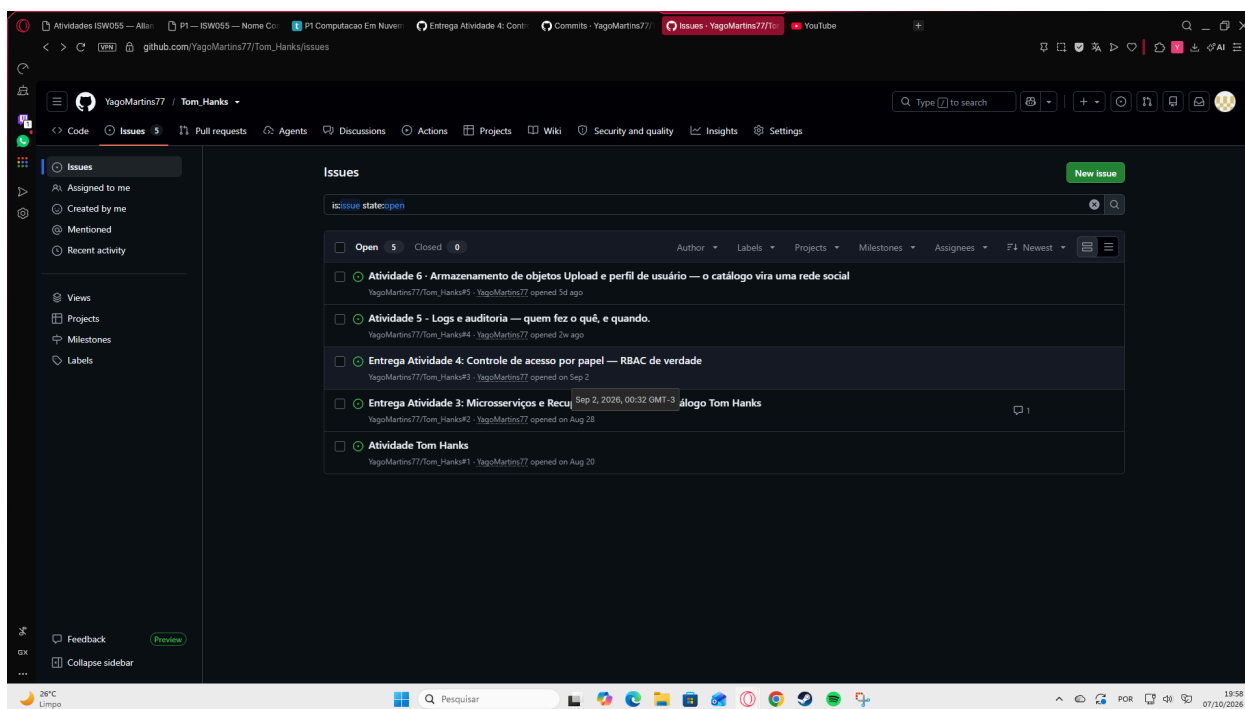
O que prova: A segurança do backend contra tentativas de IDOR ou ações não autorizadas. Quando um usuário comum tenta forçar a exclusão de um comentário alheio via requisição direta na API, o servidor barra a ação imediatamente retornando o status 403 (Forbidden).



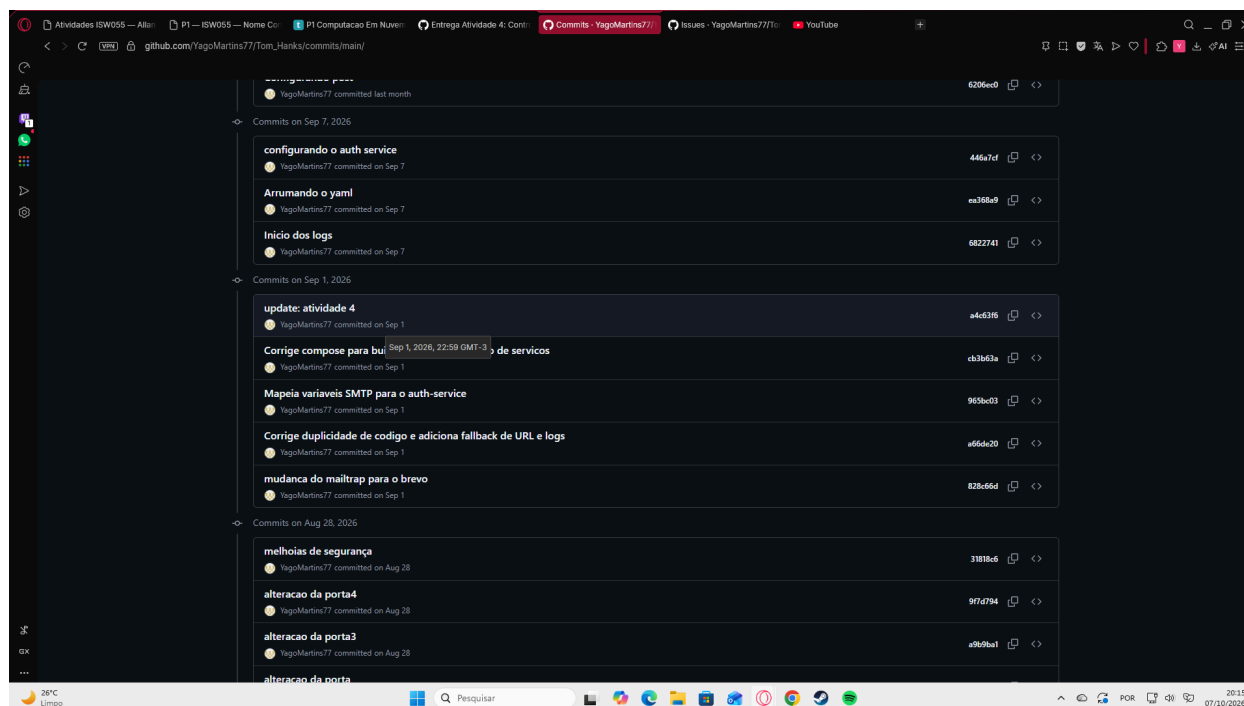
“Resposta HTTP 403 (Forbidden) ao tentar acessar rota restrita como usuário comum.”



“Painel de administração liberado e funcional para usuário com papel admin.”



“Evidência da Issue 3 no GitHub comprovando o registro e a data/hora da entrega da Atividade 4.”



“Evidência de commit no GitHub para a Atividade 4 (Controle de acesso RBAC) com hash, data e hora visíveis.”

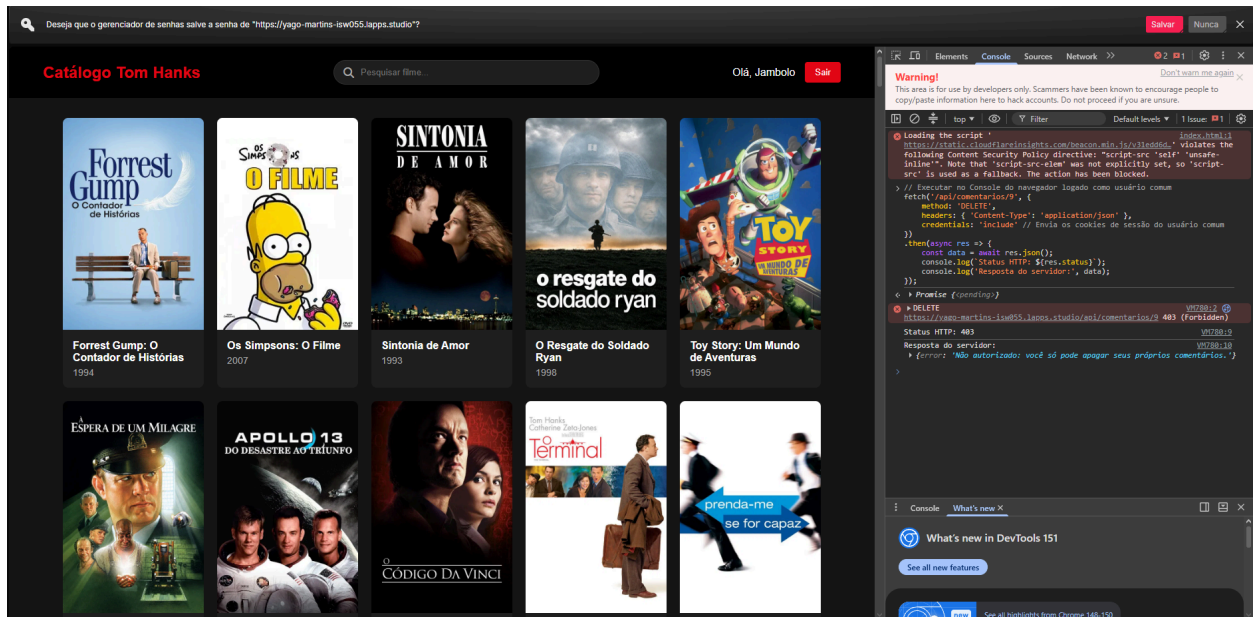
Dificuldades e como foram resolvidas. A maior dificuldade esteve na lógica da rota de validação de papéis, responsável por bloquear acessos indevidos de terceiros. A verificação estava excessivamente abrangente e acabou travando também as requisições enviadas para a API externa do TMDb. O problema foi resolvido ajustando os middlewares de verificação para atuarem estritamente nos endpoints internos protegidos da aplicação, isolando as chamadas externas.

4.5 Atividade 5 — Logs e auditoria

Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	25/09/2026 12:45
Situação	entregue
Evidência	GitHub — commit + print da consulta de logs pelo admin
Links	• Repositório • Issue 4 • README • Commits

O que foi feito. Desenvolveu-se um novo microserviço dedicado à auditoria, o log-service, rodando em um container próprio (log-service-yago) conectado à rede interna do Docker sem porta pública exposta. Optou-se pela utilização do Redis Streams devido ao seu alto desempenho para operações de escrita rápida e ordenação temporal nativa por ID. O serviço registra eventos críticos da aplicação, como tentativas de login (sucessos e falhas), logouts, cadastros, redefinição de senha, interações no catálogo (favoritar/comentar) e, fundamentalmente, tentativas de acesso negado (HTTP 403). Para viabilizar a visualização

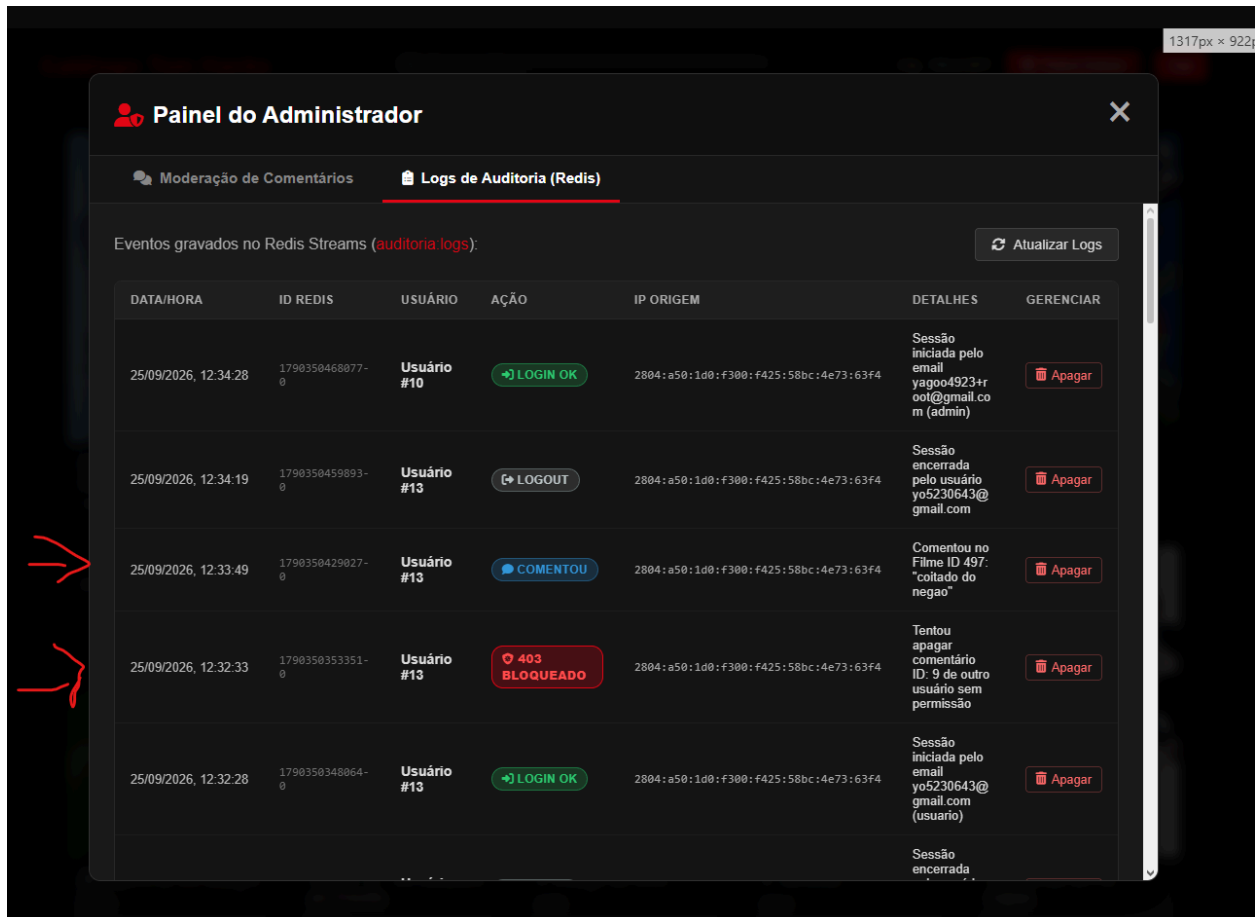
dessas métricas, criou-se uma aba de “Logs de Auditoria (Redis)” no Painel de Administração, permitindo que administradores consultem e gerenciem o histórico de eventos da aplicação via GET /api/logs.



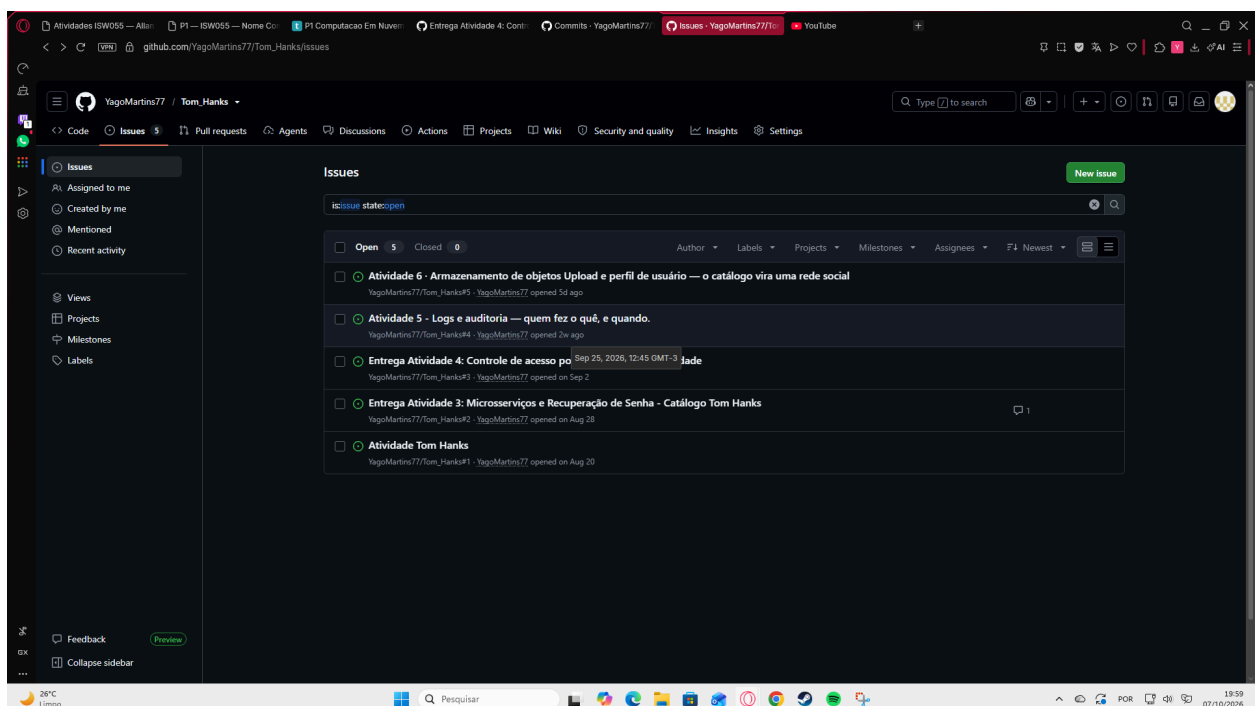
“Inicialização e conexão do microserviço log-service com o Redis Streams.”



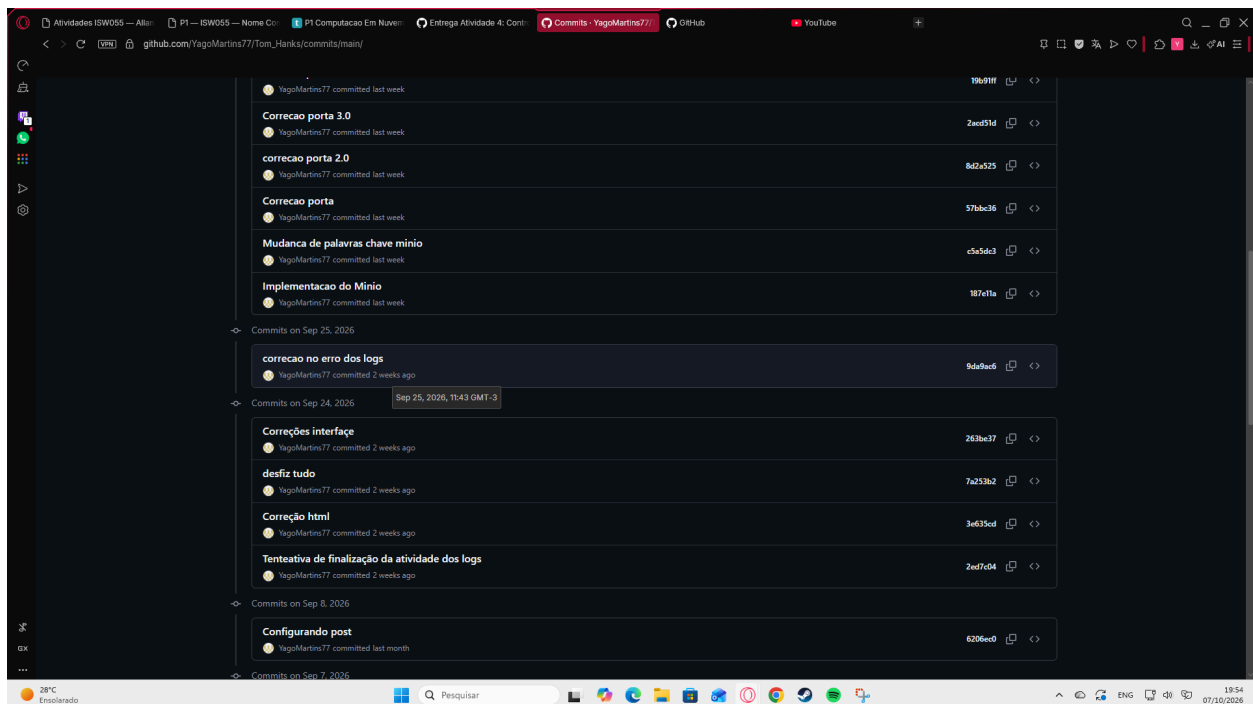
“Terminal exibindo registros de logs de auditoria gravados no Redis.”



“Aba de Logs de Auditoria no painel administrativo consultando os eventos gravados.”



“Evidência da Issue 4 no GitHub comprovando o registro e a data/hora da entrega da Atividade 5.”



“Evidência de commit no GitHub para a Atividade 5 (Log-service com Redis Streams) com hash, data e hora visíveis.”

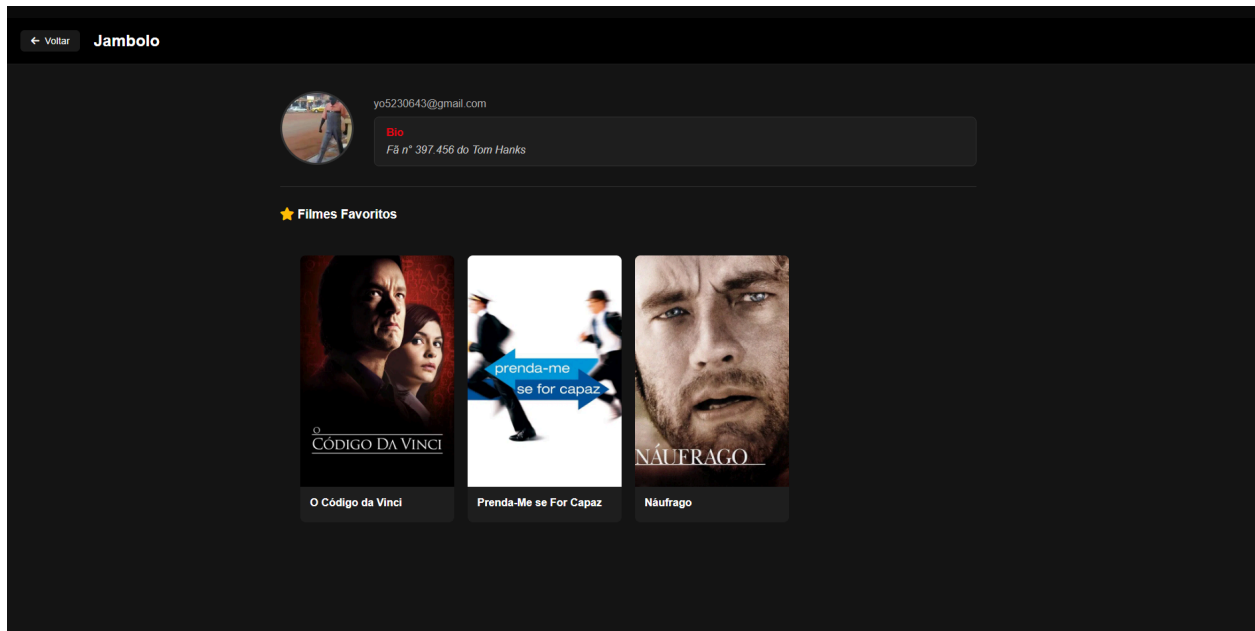
Dificuldades e como foram resolvidas. A principal dificuldade consistiu na visualização dos logs, pois inicialmente não havia uma interface clara para a consulta dos dados. A ideia de integrar um painel dedicado no modal de administrador demorou a ser concebida e, nas primeiras implementações, os logs frequentemente não eram exibidos na tela. A situação foi resolvida corrigindo a rota de repasse das requisições no catalogo-service para o log-service e ajustando a renderização assíncrona dos eventos no frontend.

4.6 Atividade 6 — Upload e perfil de usuário

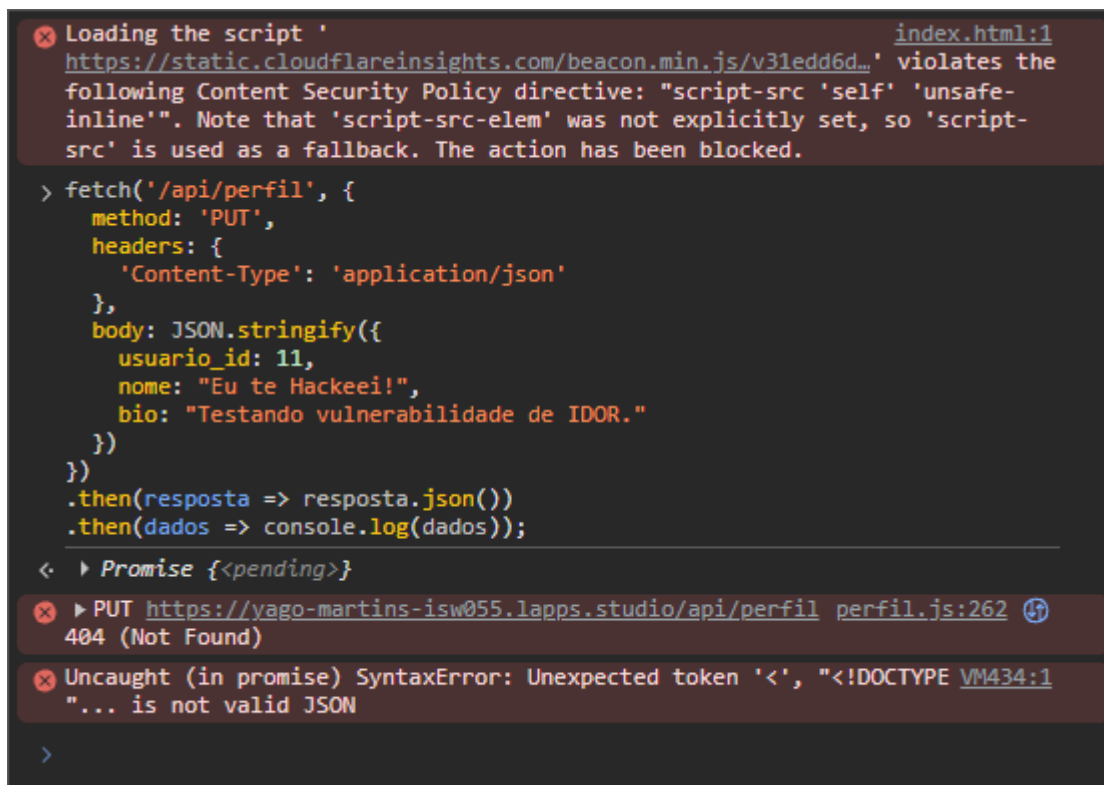
Descrição	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	02/10/2026 17:17
Situação	entregue
Evidência	GitHub — commit + print do perfil com foto
Links	• Repositório • Issue 5 • README • Commits

O que foi feito. Implementou-se a funcionalidade de gestão de perfil e avatar do usuário integrada ao servidor de armazenamento de objetos MinIO. A aplicação permite que o usuário faça o upload de sua imagem de perfil, que é processada pelo auth-service e salva em um bucket configurado com política de leitura pública (Public Read). No banco de dados relacional (MariaDB), armazena-se unicamente a referência (URL) do avatar, evitando o armazenamento de arquivos binários no banco de dados. Na interface,

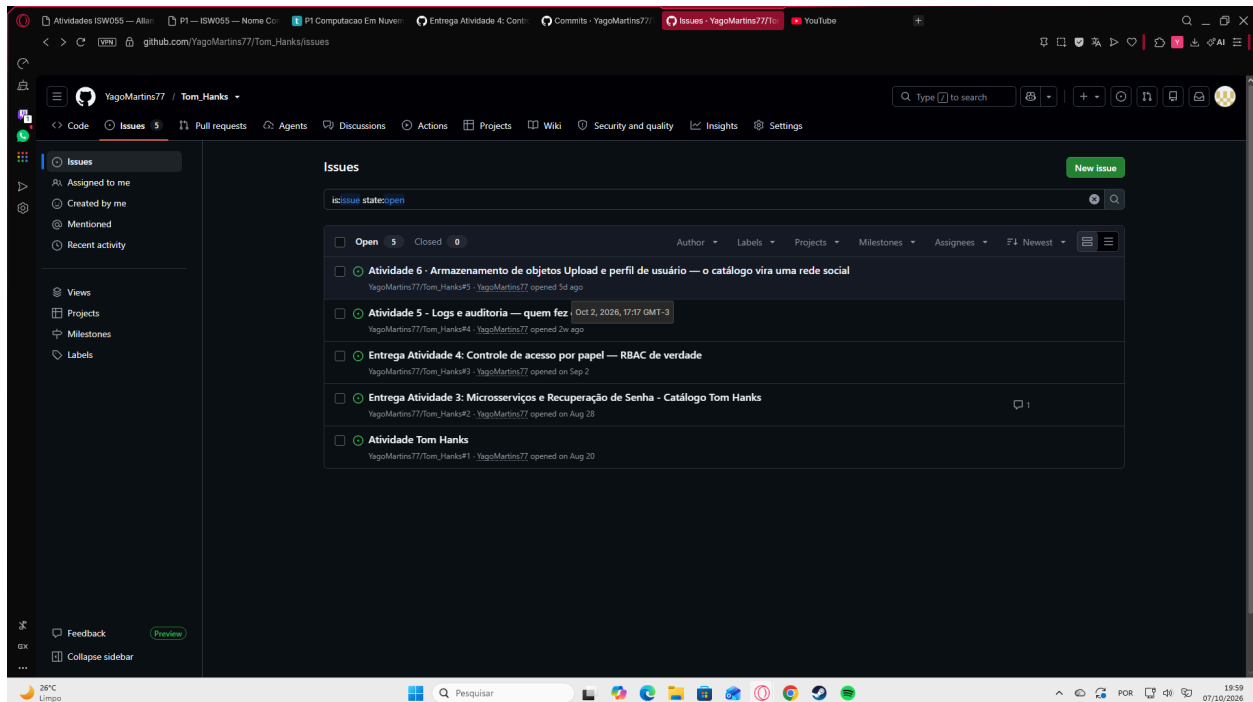
criou-se um cabeçalho com container de perfil, menu dropdown e modal para atualização das informações do usuário.



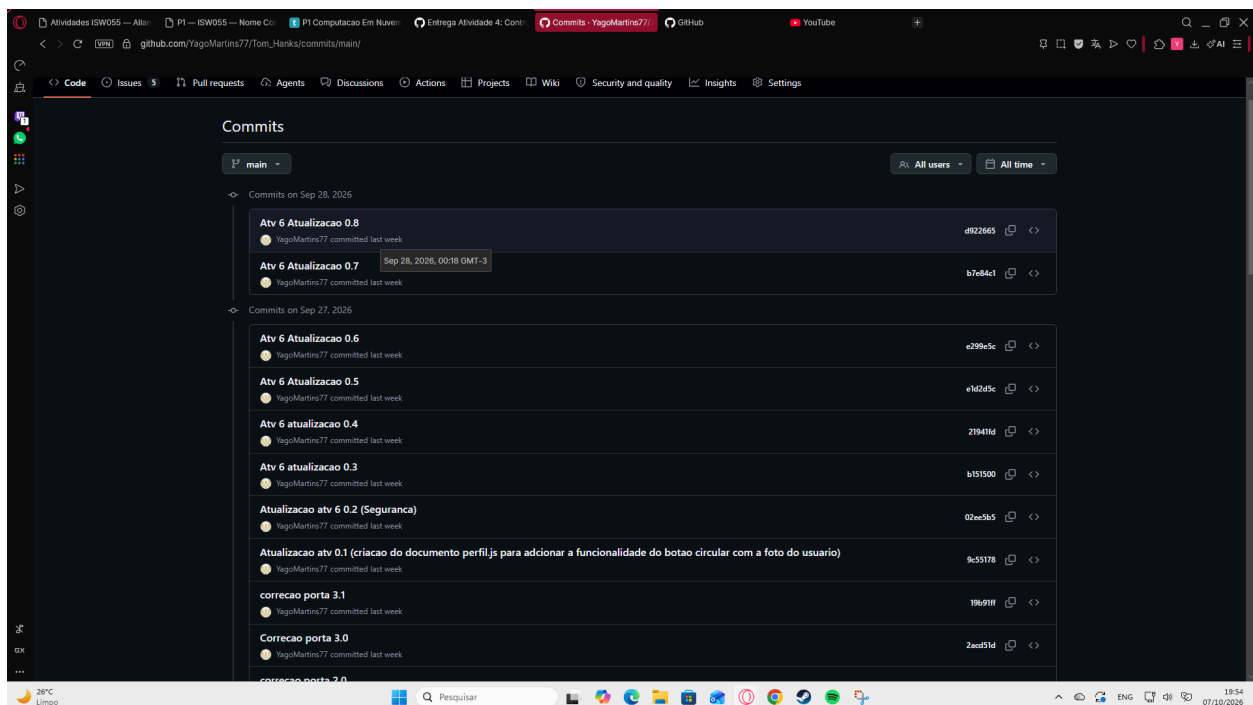
“Página de perfil exibindo o avatar do usuário carregado a partir do MinIO.”



“Modal de edição de perfil e formulário de upload de imagem de avatar.”



“Evidência da Issue 5 no GitHub comprovando o registro e a data/hora da entrega da Atividade 6.”



“Evidência de commit no GitHub para a Atividade 6 (Upload de avatar no MinIO) com hash, data e hora visíveis.”

Dificuldades e como foram resolvidas. O maior desafio nesta etapa esteve na definição da experiência visual e no design do menu do usuário. Optou-se por alinhar o visual a padrões modernos de mercado, buscando inspiração no layout limpo e intuitivo do Pinterest. Essa escolha guiou a reestruturação do cabeçalho com avatar circular, dropdown responsivo e modal de edição, superando as limitações iniciais de layout e estilo CSS.

5 Considerações finais

Ao longo do bimestre, o desenvolvimento da aplicação “Catálogo Tom Hanks” proporcionou um aprendizado prático e aprofundado sobre a arquitetura de microsserviços, containerização com Docker, desacoplamento de responsabilidades e implementação de segurança na nuvem (RBAC, auditoria e armazenamento de objetos). Compreender na prática a transição de um monólito para serviços isolados operando em rede interna fechada trouxe uma visão clara sobre os desafios reais de escalabilidade e governança de software.

O aspecto mais complexo da trajetória esteve no diagnóstico e na resolução de conflitos entre os serviços — desde problemas na rota de e-mail e bloqueios indevidos de chamadas externas por middlewares de autorização, até a renderização e consulta de logs em tempo real via Redis. Para um próximo projeto, faria diferente a fase de planejamento inicial da interface do usuário e dos contratos de API entre os microsserviços, estabelecendo esquemas e wireframes claros antes da codificação para evitar refatorações de layout e rotas.

Como principal legado para o próximo bimestre, pretendo levar a disciplina na estruturação de logs de auditoria, a organização modular do código em microsserviços e a mentalidade de segurança em camadas (Defense in Depth), garantindo que as regras de acesso e validações sejam sempre aplicadas de forma rigorosa no backend.

6 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

Yago Felipe de Oliveira Martins

Pompeia, 07/10/2026